Assignment 2: LLM Inference Chatbot with Rate Limits and Cost Control

Trainer: Umashankar K H · Client: Innomatics

Real LLM providers do not let you send unlimited traffic. In this assignment you build a chatbot whose inference layer stays inside two hard limits, **50 requests per minute (RPM)** and **100,000 tokens per minute (TPM)**, while keeping the bill as low as you can. You will get one of four sets (A, B, C or D). Each has its own traffic pattern and its own trap.

Before you start

The limits and the price card

Limit	Value	What happens if you cross it
Requests per minute	50	Provider answers HTTP 429 and the request fails
Tokens per minute (input + output)	100,000	Same: HTTP 429

Every set includes `price_card.csv`. These prices are **assumed for teaching** and are not any vendor's real rates.

Model	Input, USD per million tokens	Output, USD per million tokens
nemotron-small (assumed)	0.10	0.30
nemotron-large (assumed)	0.60	1.80
embedding-model (assumed)	0.02	0.00

```
\text{cost} = \frac{\text{input tokens} \times \text{input price} +
\text{output tokens} \times \text{output price}}{1{,}000{,}000}
```

What you must build

1. **Chatbot**: a FastAPI endpoint `/chat` plus a simple Streamlit or Gradio screen.
2. **Inference gateway**: one place every LLM call passes through. It needs an RPM limiter, a TPM limiter, a priority queue, retry with exponential backoff on 429, and token counting (estimate before the call, real usage after).
3. **Cost tracker** that uses `price_card.csv` and reports cost per request, per model and per user tier.
4. **Load simulator** that replays `traffic.csv` (each row has an `arrival_second`, so no real dates are involved).
5. **Final report** with served, queued, shed, average wait, p95 wait, total cost and what your optimisations saved.
6. **Monitoring and observability**: see the last section of this document.

Ground rules

- Run the full replay in **simulation mode** with a virtual clock and no real API calls, so ten minutes of traffic finishes in seconds. Then run **live mode** with 15 to 20 real calls to prove the gateway works against a real model.
- Count tokens with `tiktoken` as an estimate, then correct it with the `usage` numbers the API returns.
- Never drop a request silently. Every request ends as served, cached, answered by rule, or shed with a friendly message.
- Keep API keys in a .env file.

Tools you can use

What you need	Easy choice	Good to know
LLM, free to start	NVIDIA Nemotron on build.nvidia.com: `nvidia/nemotron-3-nano-30b-a3b` as the small model and `nvidia/nemotron-3-super-120b-a12b` as the large one	OpenAI-compatible API at `https://integrate.api.nvidia.com/v1`. Model names change, so check the catalogue
Embeddings	NVIDIA embedding model `nvidia/llama-3.2-nv-embedqa-1b-v2` via `NVIDIAEmbeddings`	Useful for a semantic cache

What you need	Easy choice	Good to know
Azure	Azure OpenAI or AI Foundry deployment, Azure Container Apps to host, Azure Monitor for metrics	Check the quota on your deployment, it is a real RPM and TPM limit
OpenRouter	`https://openrouter.ai/api/v1`	Good as a fallback route when the main model is throttled
Framework	LangChain (`langchain-openai`) and optionally LangGraph for routing	LangChain's `InMemoryRateLimiter` handles requests per second only, so build the token limit yourself

The easy path in six steps

1. Replay the traffic with no limits and plot requests and tokens per minute. Seeing the problem first makes the design obvious.
2. Build the limiter. A sliding window of the last 60 seconds is the simplest: keep a list of `(time, tokens)` and drop entries older than 60 seconds.
3. Add the queue. A request waits until both the request count and the token count in the window have room for it.
4. Add the optimisations your set asks for (cache, routing, trimming, shedding).
5. Add the cost tracker and compare against the baseline in your set.
6. Build the dashboard, then run live mode.

```
def can_send(now, est_tokens, window, rpm=50, tpm=100_000):
    # window = list of (timestamp, tokens) sent in the last 60 seconds
    window[:] = [(t, k) for t, k in window if now - t < 60]
    return len(window) < rpm and sum(k for _, k in window) + est_tokens <=
tpm
```

Set A: CampusBot admissions helpdesk

The story. A college runs an admissions helpdesk chatbot. On the day the merit list is published, thousands of students and parents ask the same few questions at the same moment. The model provider's limits do not care that it is a big day.

Your job. Build the chatbot and its gateway so a sudden burst is absorbed without losing a single question, then show how much money and how many LLM calls you saved.

Data you get

`traffic.csv` has 534 requests spread over 10 minutes (600 seconds). `prompts.csv` holds the question text for each request. `price_card.csv` is shared by every set.

request_id	arrival_second	user_id	user_tier	prompt_tokens	expected_output_tokens	complexity
REQA0001	0	U054	student	247	425	simple
REQA0002	0	U096	student	120	261	simple
REQA0003	2	U076	student	144	288	simple

What the traffic looks like:

- Most minutes carry 34 to 50 requests, which is right at the 50 RPM limit.
- The fifth minute (seconds 240 to 299) carries **152 requests**, about three times the limit.
- Tokens per minute stay below 100,000, so **the request limit is your problem here, not the token limit**.
- Only six different questions are asked, over and over.
- Baseline to beat: sending every request to the large model with no optimisation costs about **USD 0.36**.

Your tasks

1. Build the gateway with the 50 RPM and 100K TPM limits and a first-come-first-served queue. Replay the traffic and record the queue depth and the longest wait.
2. Add an **exact-match cache** on the normalised question text (lower-case, trimmed). Count how many LLM calls it removes.
3. Add a **semantic cache** using the NVIDIA embedding model: if a new question is more than 0.90 similar to a cached one, reuse the answer.
4. Route `simple` questions to the small model and `complex` ones to the large model.
5. Report total cost against the baseline and the share of requests served by each path (LLM, cache).

Targets

- No request is dropped.
- 95th percentile wait is 30 seconds or less.
- Total cost is at least 40 percent below the baseline.

Hints

- With six repeating questions, a cache is the biggest win by far. The first time a question is seen it goes to the LLM, and every later copy is free.
- Remember that cached requests still arrive during the burst, so the cache must be checked **before** the queue.
- Do not let two identical questions in flight call the LLM twice. Make the second one wait for the first answer.
- Use a priority of your choice for parents versus students, and explain it in the README.

Set B: BankBuddy customer support chatbot

The story. A bank offers support chat to free, premium and VIP customers. Twice during the day, a rush of customers write in after a payment app glitch. The bank has promised VIP customers fast answers, and it must never show one customer's account details to another.

Your job. Build the chatbot and a gateway that treats the three tiers differently, stays inside 50 RPM and 100K TPM, and reports what each tier costs.

Data you get

`traffic.csv` has 623 requests over 10 minutes (600 seconds). The tiers are free (378 requests), premium (177) and vip (68).

request_id	arrival_second	user_id	user_tier	prompt_tokens	expected_output_tokens	complexity
REQB0001	1	U079	premium	125	347	simple
REQB0002	1	U095	free	80	222	simple
REQB0003	2	U025	free	255	237	simple

The six questions customers ask are in `prompts.csv`:

- How do I block my debit card?
- What is the interest rate on a personal loan?
- Explain the difference between NEFT and RTGS
- Why was my UPI payment declined?
- How can I raise a chargeback dispute?

- Summarise the charges on my credit card statement

What the traffic looks like:

- Normal minutes carry 40 to 55 requests.
- Two bursts hit: the third minute (seconds 120 to 179) has **127 requests** and the eighth minute (seconds 420 to 479) has **131 requests**.
- Tokens per minute stay below 100,000, so again the request limit is the bottleneck.
- Baseline to beat: every request on the large model with no optimisation costs about **USD 0.39**.

Your tasks

1. Build a **priority queue**: vip first, then premium, then free. Add **aging**: a request that has waited more than 60 seconds is moved up one level so nobody starves.
2. Add **load shedding for the free tier**: if a free request has been waiting more than 45 seconds, answer with a friendly 'We are very busy, please try again in a minute' message and count it as shed.
3. Route `simple` to the small model and `complex` to the large model.
4. Add a cache, but decide carefully what is safe to cache (see the first hint).
5. Produce a cost report by tier and by model, with cost per served request.

Targets

- VIP p95 wait is 10 seconds or less. Premium p95 wait is 30 seconds or less.
- No more than 15 percent of free requests are shed.
- Total cost is at least 40 percent below the baseline.

Hints

- Two of the six questions are about the customer's own account ('Why was my UPI payment declined?' and 'Summarise the charges on my credit card statement'). **Never serve those from a shared cache.** Say in your README how you decided.
- A Python `heapq` with tuples of (priority, arrival_second, request_id) is enough for the queue.
- Run the replay twice, once with only a FIFO queue and once with your priority queue, and show both p95 waits side by side. That is your proof the priority logic works.

- Stretch: limit each `user_id` to 5 requests per minute so one customer cannot hog the queue.

Set C: FestiveCart sale-day shopping assistant

The story. An online store runs a festival sale. In two peak minutes the chat assistant gets almost four times the traffic it can send to the model, and **even outside the peaks it is over the limit**. No amount of clever queueing will push 941 questions through a 50 RPM pipe in ten minutes, so the design has to change.

Your job. Build an assistant that answers most questions *without* calling the LLM, keeps members happy first, and degrades politely for guests.

Data you get

`traffic.csv` has 941 requests over 10 minutes (600 seconds). The tiers are guest (604 requests) and member (337).

request_id	arrival_second	user_id	user_tier	prompt_tokens	expected_output_tokens	complexity
REQC0001	1	U029	guest	114	286	complex
REQC0002	1	U038	guest	50	376	simple
REQC0003	3	U007	guest	137	225	simple

The six questions in `prompts.csv`:

- Where is my order?
- How do I return a product?
- Is the sale price applicable on my cart?
- Why was my coupon rejected?
- Compare these two phones for me
- Explain the exchange offer terms

What the traffic looks like:

- Requests per minute across the ten minutes: 74, 83, 62, 64, 76, **198**, **192**, 50, 72, 70.
- Nine of the ten minutes are above 50 RPM. Even a perfect limiter can send only about 500 requests to the model in ten minutes.
- Tokens per minute stay below 100,000.

- Baseline to beat: every request on the large model costs about **USD 0.49**, and cannot actually be served inside the limits.

Your tasks

1. **Prove the problem.** Replay with a plain FIFO queue and show the queue growing without limit and the waits exploding.
2. **Make a routing table** with one row per question: which path answers it (rule or tool, cache, small model, large model, or shed) and why.
3. **Build the no-LLM paths.** For example, 'Where is my order?' can call a mock order-status tool and fill a template. Cart and coupon questions can call a mock cart service.
4. **Add a cache** for answers that are the same for everyone.
5. **Add priority and shedding**: members before guests, and a guest who has waited more than 30 seconds gets a friendly 'busy, please retry' reply.
6. Report the **share of requests answered by each path**, wait times per tier, and cost.

Targets

- At least 90 percent of requests get a real answer (rule, cache or LLM), not a shed message.
- Member p95 wait is 20 seconds or less.
- Zero requests rejected by the provider (no 429s in your simulation).
- Total cost is at least 50 percent below the baseline.

Hints

- Think about which questions need a language model at all. A coupon rejection is almost always one of a handful of reasons a program can look up.
- Only questions that are the same for everybody belong in the shared cache. Comparing two phones depends on which phones, so it should not be cached.
- A friendly shed message is a design feature, not a failure. Write it well.
- Send your routing table and the final share-by-path numbers in your README. A reviewer should be able to read your whole strategy in one table.

Set D: PolicyPal internal policy assistant

The story. A company wants employees and managers to ask questions about its long policy documents. To be accurate, the first version pastes the whole policy into every prompt. The prompts are huge, and this time **the token limit breaks before the request limit does**.

Your job. Make the assistant fit inside 100K tokens per minute by sending less text, and show how many tokens and how much money that saves.

Data you get

`traffic.csv` has 279 requests over 10 minutes (600 seconds). The tiers are employee (225 requests) and manager (54). Almost every request is `complex`.

request_id	arrival_second	user_id	user_tier	prompt_tokens	expected_output_tokens	complexity
REQD0001	1	U061	employee	2101	686	complex
REQD0002	1	U005	employee	3465	793	complex
REQD0003	3	U068	employee	4025	479	complex

The five questions in `prompts.csv`:

- Summarise the leave policy section for contract employees
- What is the reimbursement process for travel above 50000?
- Compare the old and new expense policy documents
- Explain the data security policy for remote work
- List all approvals required for vendor onboarding

What the traffic looks like:

- Requests per minute peak at 47, so the **request limit is never crossed**.
- Prompts range from 1,800 to 6,500 tokens and total about **891,000 input tokens** for the run.
- In minutes 4, 5, 9 and 10 (seconds 180 to 299 and 480 to 599) the tokens per minute go above 100,000. The peak minute reaches about **177,000**.
- Baseline to beat: every request on the large model with the full document costs about **USD 0.84**.

Your tasks

1. Replay the traffic and plot requests per minute and tokens per minute together, to show that tokens, not requests, are the limit.
2. Make your limiter **reserve tokens before sending**: reserve the prompt tokens plus the expected output tokens, and give back the unused part when the real answer arrives.
3. **Trim the context.** In a real build you would embed the document in chunks with the NVIDIA embedding model and send only the top three chunks. For the simulation, assume that retrieval lets you send the question plus three chunks of about 400 tokens, so cap every prompt at **1,300 tokens**.
4. Cap the answer length (for example `max_tokens` of 400) and measure what it saves.
5. Give managers priority over employees in the queue.
6. Report input tokens before and after, cost before and after, and wait times per tier.

Targets

- Zero requests rejected by the provider, and every minute stays at or below 100,000 tokens.
- Input tokens cut by at least 50 percent.
- Total cost at least 40 percent below the baseline.
- 95th percentile wait of 20 seconds or less.

Hints

- The limiter only needs a token budget per sliding minute. The request count rarely matters here, but keep it anyway so the gateway is complete.
- After trimming, **output tokens become the bigger part of the bill**. That is why capping the answer length matters.
- A request that is bigger than the whole per-minute budget could never be sent. Check that case and decide what to do.
- In your README, explain what accuracy risk comes with trimming context and how you would measure it.

Monitoring, observability and marking

A gateway you cannot see is a gateway that fails at the worst moment. Whatever set you got, add monitoring to your gateway and show it working during the replay.

What to measure

Metric	How to get it
Requests and tokens in the last 60 seconds, as a percentage of 50 RPM and 100K TPM	Your limiter's sliding window
Queue depth over time	Count of waiting requests, sampled every second of simulated time
Wait time, p50 and p95, per tier	Start of service minus arrival second
Rejections (429) and retries	Counters in the gateway
Cache hit rate and share of requests by path (LLM, cache, rule, shed)	Counters per path
Cost per minute, cumulative cost, cost per model and per tier	Real or estimated tokens times the price card
LLM call latency and tokens in and out	From live mode responses

How to do it, the easy way

- **Simplest:** write one JSON line per event to `metrics.jsonl` and draw the charts with a small Streamlit and Plotly page.
- **Prometheus and Grafana:** run both with Docker Compose and expose metrics from Python. The core of it looks like this:

```
from prometheus_client import Counter, Histogram, Gauge, start_http_server

LLM_TOKENS = Counter('llm_tokens_total', 'Tokens used', ['direction',
'model'])
WAIT = Histogram('queue_wait_seconds', 'Time in queue', ['tier'])
QUEUE = Gauge('queue_depth', 'Requests waiting')
start_http_server(8000)
```

- **Langfuse or LangSmith:** trace the 15 to 20 live calls to see prompts, latency, tokens and cost per call.
- **Azure:** send the same metrics to Azure Monitor if you host the gateway on Azure.

The dashboard (one page)

Draw requests per minute and tokens per minute as lines with the 50 and 100,000 limits marked. Add queue depth over time, cumulative cost, wait times by tier, and the share of requests by path. A reviewer should see in ten seconds whether you stayed inside the limits.

Alerts (at least two)

For example: requests or tokens above 80 percent of the limit for three checks in a row, p95 wait above your target, projected cost above a budget you set, or any 429 at all. Show how the alert fires in the replay.

Prove it works

Replay once with your protections switched off and once with them on, and show the two dashboards side by side.

How you will be marked

Area	Marks
Limiter correctness (RPM and TPM, no 429s in simulation)	25
Queue, priorities and shedding	15
Cost control (routing, cache, trimming) against the baseline	20
Cost tracking and final report	10
Monitoring and observability (dashboard, alerts)	20
Live mode, code quality, README	10
Total	**100**